Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 00 (2026) 000–000

Procedia Computer Science

www.elsevier.com/locate/procedia

30th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2026)

Chunking Methods on Retrieval-Augmented Generation – Effectiveness Evaluation Against Computational Cost and Limitations

Mateusz Śmigielski[a], Michał Rajkowski[a], Mateusz Zbrocki[a], Michał Bernacki-Janson[a], Karol Kunicki[a], Julianna Godziszewska[a], Maciej Piasecki[a], Konrad Wojtasik[a]

[a]*Department of Artificial Intelligence, Faculty of Information and Communication Technology, Wrocław University of Science and Technology, Wrocław 50-370, Poland*

Abstract

Retrieval-Augmented Generation (RAG) has demonstrated significant capabilities in enhancing the performance of Large Language Models (LLMs). One of the key tasks in RAG systems is the chunking process. Traditionally, fixed-size chunking and semantic chunking have been the standard approaches. However, interest in chunking strategies has been increasing, leading to a growing number of proposed methods that often claim improved performance over these conventional techniques. Many of these approaches are tailored to specific use cases and data types, with limited evidence of their effectiveness across diverse scenarios. As a result, it remains challenging to directly compare different techniques and assess their relative strengths. To the best of our knowledge, this study is the first to systematically evaluate the effectiveness of a wide range of chunking methods and emphasize the underlying challenges of chunking strategies in RAG systems. While chunking is commonly treated as a simple preprocessing step, we show that it introduces a range of impactful and often overlooked issues.

Keywords: Large Language Models; Retrieval Augmented Generation; Chunking methods;

1. Introduction

Retrieval Augmented Generation (RAG) has been proposed as a response to several Large Language Models (LLMs) problems like accessing, manipulating and updating knowledge or providing provenance for their decision [16]. A standard RAG pipeline initially splits documents, especially the longer ones, into smaller units called *chunks*.

* Corresponding author. Tel.: +0-000-000-0000
E-mail address: author@institute.xxx

Identifying the chunks most pertinent to a query and supplying them to the LLM – the answer generator should allow for obtaining answers that are accurate and well aligned with the context. Chunks granularity is often reported to have a notable influence on both the retrieval stage and generations tasks [29, 23]. Large chunks may contain irrelevant information, while small chunks may lack information, which may lead to hallucination and distract the LLM ability to extract accurate key information [30]. Additionally, [28] demonstrate that chunking methods significantly impact retrieval performance, although their study evaluates only a limited subset of the methods. Recently, the importance of chunking has become increasingly recognized, as evidenced by emerging surveys and benchmarking studies dedicated specifically to document segmentation strategies. Zhou et al. [36] highlight the diversity of document chunking methods in dense retrieval and emphasize the need for a systematic taxonomy to understand their design space. They categorize existing strategies based on segmentation approaches and embedding paradigms, providing a framework for comparing and evaluating chunking techniques. Renyi Qu et al. [20] evaluate the effectiveness of semantic chunking and demonstrate that its computational costs are not justified by consistent performance gains. The study compares semantic chunking with simpler fixed-length baselines and analyzes trade-offs between retrieval quality and efficiency. The results indicate that while semantic chunking can improve contextual coherence and, in certain scenarios, retrieval performance, the gains are not always proportional to the increased computational cost. This suggests that the adoption of advanced chunking techniques should be guided by practical constraints and system requirements, especially in large-scale or latency-sensitive applications. The relevance of chunking has also been explicitly acknowledged in medical applications, where retrieval errors may directly affect clinical reasoning. In a recent study, Gomez-Cabello et al. [7] conduct a comparative evaluation of chunking strategies within RAG systems designed for clinical decision support. Their analysis demonstrates that chunking methodology significantly influences RAG-LLM performance for clinical decision support. However, it should be noted that the scope of chunking strategies considered in that study was relatively limited and did not cover the full spectrum of existing segmentation methods.

Our contribution in this study is threefold. First, we present an evaluation framework designed to enable the systematic comparison of a broad range of chunking methods across diverse tasks and dataset types. Second, we identify several practical and methodological challenges, referred to as efficiency limitations, including high chunking time and implementation fragility. Third, we show that more computationally expensive chunking methods do not yield meaningful effectiveness improvements while introducing substantially higher computational overhead. To facilitate systematic comparison and reproducible analysis, we provide a unified evaluation framework available at https://github.com/ApriiM/Chunking-Research.

The remainder of this paper is structured as follows. First, we provide a review of existing document chunking methods, outlining their key characteristics and differences. We then describe the chunk preparation pipeline, the dataset selection process and the evaluation methodology employed in our study. This is followed by a presentation and analysis of the experimental results, highlighting performance differences, limitations, and inconsistencies across methods. Finally, we summarize our findings and discuss potential directions for future research.

2. Related works

2.1. Theoretical Background

Text segmentation, often referred to as chunking, is a preprocessing step that transforms a continuous text stream into discrete text segments.

Formally, let us define a document D as a sequence of atomic units (tokens) $D = \langle t_1, t_2, \ldots, t_N \rangle$. The goal of the chunking function f_θ is to map D into a set of segments $C = \{c_1, c_2, \ldots, c_m\}$, such that: $c_i = \langle t_{start_i}, \ldots, t_{end_i} \rangle$.

While strictly disjoint segmentation is possible, overlapping segmentation is often employed to preserve boundary contexts.

The Granularity-Context Trade-off. The fundamental theoretical challenge is maximizing the probability of generating the correct answer a given a query q. This introduces a trade-off between *granularity* and *context preservation*.

Chunking in the RAG Pipeline. In the vector space model, chunking defines the granularity of the search index. Let $\mathcal{E}$ be an embedding function (e.g., BERT-based encoder) mapping a text segment to a vector $v \in \mathbb{R}^d$ [13]. The retrieval process identifies a subset $C_{retrieved} \subset C$ by minimizing the distance metric:

$$C_{retrieved} = \arg\operatorname*{topk}_{c \in C} \left(\frac{\mathcal{E}(q) \cdot \mathcal{E}(c)}{\|\mathcal{E}(q)\| \|\mathcal{E}(c)\|} \right) \tag{1}$$

Ineffective segmentation leads to a mismatch in the embedding space, where the vector representation $\mathcal{E}(c)$ is dominated by irrelevant content, causing semantic misalignment with the query $\mathcal{E}(q)$ [22].

2.2. Chunking methods

The foundational strategies for text segmentation rely on surface-level features such as character count or punctuation, ignoring semantic content. *Fixed-size Chunking* divides text into segments of predetermined length, often with overlapping windows to reduce mid-sentence splits, but it frequently fragments semantic dependencies [2]. *Sentence/Paragraph Splitting* preserves linguistic units like sentences or paragraphs, though chunk sizes vary widely, which can limit context for embedding models. *Recursive Character Splitting* uses a hierarchy of separators (paragraphs, lines, spaces) to iteratively split oversized segments, preserving semantic coherence when possible. *Entropy-Optimized Dynamic Text Segmentation (EDTS)* determines chunk boundaries by analyzing lexical homogeneity via information entropy [27]. *Semantic Chunking* detects topic transitions by monitoring similarity drops between consecutive sentences, aligning chunks with thematic coherence but incurring high computational cost and threshold selection challenges [12].

LLM-based chunkers leverage language models to produce semantically coherent text segments. *LumberChunker* [6] splits documents into paragraphs and aggregates them into groups passed to an LLM, which identifies points of semantic shift to finalize chunks. Evaluated on GutenQA, it outperforms Semantic, Recursive, and Proposition-Level Chunking, but incurs high computational cost and is mainly suited for narrative texts. *Propositions* [3] move from chunk-level to finer-grained atomic units encapsulating distinct factoids, automatically generated via a propositionizer and indexed for retrieval, related to prior notions of factoids and information nuggets [18, 19]. *Pseudo-Instruction for Document Chunking* [30] groups consecutive sentences based on semantic similarity to a document summary, forming alternating segments of highly related versus less related content. *HiChunk* [17] detects Global Chunk Points with a fine-tuned LLM, assigning hierarchical levels and iteratively processing fragments to handle long documents; a fixed-size post-chunking step and Auto-Merge Retrieval improve uniformity and query-aligned retrieval. *Logits-Guided Multi-Granular Chunker (LGMGC)* combines Logits-Guided Chunking, which uses LLM token probabilities to detect semantic endpoints, with Multi-Granular Chunking, recursively subdividing parent chunks into smaller levels, ranking all chunks by similarity to the query for final retrieval. *Meta-Chunking* framework [33] introduces Perplexity Chunking and Margin Sampling Chunking to enhance the semantic integrity and contextual coherence of chunks. Additionally, it integrates meta-chunking with dynamic merging and employs a global information compensation mechanism via hierarchical summarization and multi-stage chunk rewriting to repair semantic discontinuities caused by segmentation.

Adaptive approaches avoid committing to a single segmentation strategy. *Mix-of-Granularity (MoG)* [35] pre-segments documents into multiple granularity levels, and a query-conditioned routing module selects which level to use for retrieval, aligning chunk length and specificity with query needs. *Mixtures of Chunking Learners (MoC)* [32] combines multiple chunking models to generate alternative segmentations, after which a router selects a meta-chunker model based on chunk-quality metrics. The chosen SLM-based meta-chunker is trained to predict only structured chunk boundaries, masking central content during training, which allows efficient reconstruction from the original document while reducing computational overhead. This design highlights the complementary strengths of different chunking strategies and improves semantic alignment with queries.

Clustering-based chunking treats segmentation as a grouping task, identifying semantically similar units rather than cutting text arbitrarily. *Sequential Hierarchical Agglomerative Chunking* [20] merges adjacent sentences based on semantic similarity while enforcing a strict structural constraint, ensuring narrative continuity without tuning positional weights or predefining the number of chunks. *Max-Min Semantic Chunking* [14] uses a greedy strategy that sequentially adds sentences to a chunk only if their maximum similarity exceeds an adaptive threshold derived from the chunk's minimum coherence, dynamically controlling chunk growth.

Recent structural chunkers leverage document layout and element types (titles, tables, figures) detected by vision models or object detectors to guide segmentation. *AutoChunker* converts documents into Markdown, splits into sentences, and uses an LLM to merge or discard sentences, producing hierarchical chunks based on semantic structure [10]. *cAST* segments code using Abstract Syntax Trees, producing chunks aligned with syntactic boundaries [31]. *S2 Chunking* builds a weighted graph over elements using spatial proximity and semantic similarity, then applies spectral clustering to form chunks, enforcing a maximum token length per chunk [25].

An interesting alternative approach for chunked document contextualization is *Late Chunking* [8]. Instead of adding extra context text to each chunk (e.g., summaries or surrounding passages), Late Chunking adds context at the embedding level. A long-context embedding model first encodes tokens from the entire document, producing token-level representations. Then, chunk boundaries are computed on the original text, and the token embeddings within each chunk are pooled into a single vector.

Additionally, some classical chunking approaches from earlier NLP research have been largely overlooked in the context of RAG. For instance, *TextTiling* [9] was an early method designed to detect topical shifts in texts based on lexical cohesion. Despite their historical relevance, these techniques are rarely included in modern evaluations of chunking strategies for RAG. As a result, it remains unclear whether such linguistically motivated approaches could complement or even outperform certain contemporary embedding- or model-based chunking methods.

3. Methodology

To comprehensively cover different approaches to chunking, we selected a diverse set of chunking methods representing classical (Fixed Size, TextTiling), semantic (Recursive Semantic), clustering-based (Sequential HAC Chunker, Max Min Chunker), graph-based (GraphSeg), and LLM-based (Lumberchunker) approaches. Default hyperparameters were taken from the original publications introducing the methods or, when not explicitly specified, from the corresponding official code repositories, with the exception of Lumberchunker, where we used GPT-OSS-20B. Additionally, we modified the TextTiling implementation so that chunk boundaries were aligned to the nearest sentence rather than the nearest paragraph. Preliminary experiments indicated that paragraph-level normalization frequently resulted in no effective segmentation at all, often returning the original documents without introducing meaningful chunk boundaries across nearly all evaluated datasets. In contrast, sentence-level alignment yielded substantially more coherent and stable chunks. Since Fixed-Size Chunking is a simple heuristic rather than a formally grounded method, we used a chunk size of 512 (i.e. quite common choice) with an overlap of 50. Consequently, chunk sizes were not normalized across methods, as we wanted each method to freely adapt its chunking strategy according to its design assumptions. The resulting configurations were then used to segment documents across the selected datasets. Datasets were selected to fulfill requirements and cover a wide range of domains to enable reliable evaluation of the chunking methods. Additionally, due to the lack of sufficiently challenging datasets, we merged each of the three selected datasets into a single massive document to create stress-test conditions for the evaluated methods (Appendix A). In order to cope with the complexity of the problem, we had to delimit the scope of our experimental study, imposing a 48-hour time limit for each chunking process, as we considered any method exceeding this duration to introduce excessive computational overhead. At this stage, we also propose a principled evaluation framework for the qualitative analysis of the generated chunks that we include in our code repository. After preparing the chunks, we designed two experiments corresponding to two different stages of the RAG pipeline.

3.1. Evidence retrieval

The first experiment evaluated evidence retrieval, focusing on the effectiveness of chunking methods in retrieving relevant chunks for a given query. A chunk was considered relevant if it overlapped with the span of the extractive answer. In cases where no extractive answer was available, all chunks originating from the relevant document were treated as relevant. To ensure a fair comparison across methods, we evaluated the number of relevant chunks included in the top-k retrieved results using the unified Polish Information Retrieval Benchmark (PIRB) [4] with accuracy and recall as the evaluation metrics. During the retrieval evaluation phase, the bge-m3 model was employed as the primary retriever, while bge-reranker-v2-m3 served as the reranking model.

3.2. End-to-end RAG Answer Generation

The second experiment evaluated the impact of chunking methods on the end-to-end RAG system and the quality of LLM-generated answers. We employed the GPT-OSS-20B model as the generative model. For each query, the top-5 retrieved chunks were provided as contextual input with a restriction to 4,000 tokens. The generated responses were then evaluated against ground-truth answers using an LLM-as-a-judge approach [34], with GPT-OSS-20B serving as the judge model. The evaluation was conducted using a five-point Likert scale, where scores ranged from 1 (poor answer quality and relevance) to 5 (highly accurate and relevant answer). The experiments were conducted on a subset of the original datasets due to missing answers in some datasets. Additionally, the TextTiling method was excluded from the comparison as it relied on a modified implementation that was not directly comparable to the other chunking approaches under a unified experimental setup.

4. Experiments and Results

For each chunker, text segmentation was performed on all datasets included in the study. The segmentation process was conducted in parallel on 20 nodes with the following hardware specifications: CPU: 2x Intel Xeon Platinum 8462Y (32 cores, 2.8 GHz), GPU: 4x Nvidia H100 96GB, 16896 CUDA cores, 1980 MHz, RAM: 1006 GB 4800 MT/s ECC DDR5. So, approximately, each was run on 1/4 of a node.

Table 1. Accuracy@5 for evidence retrieval. T - chunking procedure exceeded the allotted execution time; S - The underlying spaCy library triggered an error due to excessive temporary memory requirements.

Dataset	DenseX	Recursive semantic	Fixed-size	Sequential HAC	Text Tiling	Max-min	GraphSeg	Lumberchunker
GutenQA	T	**92.42**	87.86	71.45	82.12	82.72	88.53	T
GutenQA merged	T	**89.70**	87.49	65.92	77.66	78.23	S	T
LiteraryQA	T	**87.59**	85.91	75.73	82.81	83.40	S	T
NQ	97.33	**99.00**	98.00	**99.00**	**99.00**	**99.00**	98.67	T
NovelQA	T	**91.61**	90.10	82.90	88.07	88.53	S	T
PoQuAD	25.80	**97.68**	96.39	88.74	96.36	93.58	97.45	97.05
PoQuAD merged	T	93.11	94.67	88.86	91.74	93.62	S	**97.48**
Qasper	49.55	49.97	51.00	51.27	50.52	**51.48**	49.62	50.10
SQuAD	85.13	96.87	96.24	94.60	95.81	96.24	**97.34**	97.10
TriviaQA	79.50	**91.80**	87.30	78.30	85.10	86.30	89.50	T
TriviaQA merged	77.30	**93.20**	89.80	84.20	85.40	90.10	S	T
Average	69.10	**89.36**	87.71	80.09	84.96	85.75	86.85	85.44

Tab. 1 and 2 present the results of the first experiment – evidence retrieval, respectively: Accuracy@5 and Recall@10, across a total of 88 experimental configurations. The best-performing results for each dataset are in bold.

Across experiments, a substantial number of configurations did not produce usable outputs. We identified them as practical and methodological issues. With the exception of TextTiling and HAC Semantic Chunker, our experiments utilized the original implementations provided by the authors of the respective chunking methods. Consequently, any issues related to code optimization, execution bottlenecks, or structural failures reflect the current state of these specific implementations. When an error or failure occurred, the corresponding configuration was excluded from further downstream evaluation, as resolving third-party implementation inefficiencies falls outside the scope of this comparative study. The encountered failures are denoted in the tables using the following markers: The marker “T” indicates severe scalability and efficiency limitations; in many cases, the chunking process required more than 48 hours to complete. The marker ”S” further reveals implementation fragility, where spaCy-related errors prevented the processing of long documents exceeding one million characters.

An analysis of these failures reveals significant robustness issues across several approaches. GraphSeg performs well when execution succeeds, showing competitive retrieval quality across both metrics, but its applicability is lim-

Table 2. Recall@10 for evidence retrieval. T - chunking procedure exceeded the allotted execution time; S - The underlying spaCy library triggered an error due to excessive temporary memory requirements.;

Dataset	DenseX	Recursive semantic	Fixed-size	Sequential HAC	Text Tiling	Max-min	GraphSeg	Lumberchunker
GutenQA	T	**67.62**	39.65	11.31	23.48	26.58	47.74	T
GutenQA merged	T	**53.72**	36.57	10.92	20.34	25.50	S	T
LiteraryQA	T	**7.18**	4.53	0.62	2.02	1.81	S	T
NQ	4.28	**30.18**	16.69	5.11	13.08	10.34	18.02	T
NovelQA	T	**0.92**	0.44	0.07	0.19	0.22	S	T
PoQuAD	24.01	**98.31**	96.26	89.86	96.95	94.69	98.01	97.66
PoQuAD merged	T	82.85	82.81	79.25	82.02	83.55	S	**87.05**
Qasper	11.71	46.14	34.37	18.25	26.98	35.66	**48.95**	40.81
SQuAD	66.28	85.81	81.81	79.84	81.79	84.24	85.98	**87.14**
TriviaQA	38.42	**78.66**	64.90	49.95	61.23	62.05	71.78	T
TriviaQA merged	19.90	**40.52**	34.18	27.19	30.27	33.82	S	T
Average	27.43	53.81	44.75	33.85	39.85	41.68	61.75	**78.16**

Table 3. Chunking time per dataset and method

Dataset	DenseX	Recursive semantic	Fixed-size	Sequential HAC	Text Tiling	Max-min	GraphSeg	Lumberchunker
GutenQA	-	10.99m	<1s	8.72m	2.22m	9.18m	1.13h	-
GutenQA merged	-	11.89m	<1s	3.27m	6.37m	4.09m	-	-
LiteraryQA	-	9.91m	<1s	3.19m	2.69m	4.16m	-	-
NQ	30.79h	2.98m	<1s	2.58m	36s	2.84m	11.68h	-
NovelQA	-	11.02m	<1s	3.18m	2.66m	4.03m	-	-
PoQuAD	3.05h	43s	<1s	17s	23s	17s	47s	9.94h
PoQuAD merged	-	22s	<1s	4s	24s	5s	-	4.27h
Qasper	34.34m	15s	<1s	5s	7s	4s	17s	1.90h
SQuAD	5.47h	39s	<1s	7s	16s	9s	32.55m	17.36h
TriviaQA	25.39h	2.60m	<1s	54s	38s	1.10m	5.17h	-
TriviaQA merged	25.03h	2.51m	<1s	39s	48s	50s	-	-
Average	15.05h	4.90m	**<1s**	2.10m	1.56m	2.44m	3.09h	8.37h

ited by memory-related failures in the underlying implementation. Lumberchunker is highly inconsistent, occasionally achieving excellent retrieval performance, but failing on many datasets due to time constraints, which significantly reduces its overall robustness. These limitations are largely attributable to its reliance on an underlying LLM, whose behavior can lead to overly conservative or inconsistent segmentation decisions. DenseX exhibited lower performance across most experimental settings, frequently failing entirely or producing low-quality outputs when execution was possible. Overall, only Recursive Semantic, Fixed-size, Max-min, Text Tiling and Sequential HAC consistently provide sufficient robustness for full evaluation coverage, with the first two offering the best trade-off between stability and retrieval effectiveness.

Table 3 details the chunking execution times across the experiments. The results clearly establish Fixed-size chunking as the most computationally efficient method, significantly outperforming all other approaches. Conversely, DenseX emerged as the slowest method by a substantial margin. The remaining techniques fall within an intermediate range: Sequential HAC, Max-min, Text Tiling, and Recursive Semantic exhibited comparable, moderate runtimes. GraphSeg and Lumberchunker, however, required the longest execution times after DenseX, indicating considerably higher computational overhead.

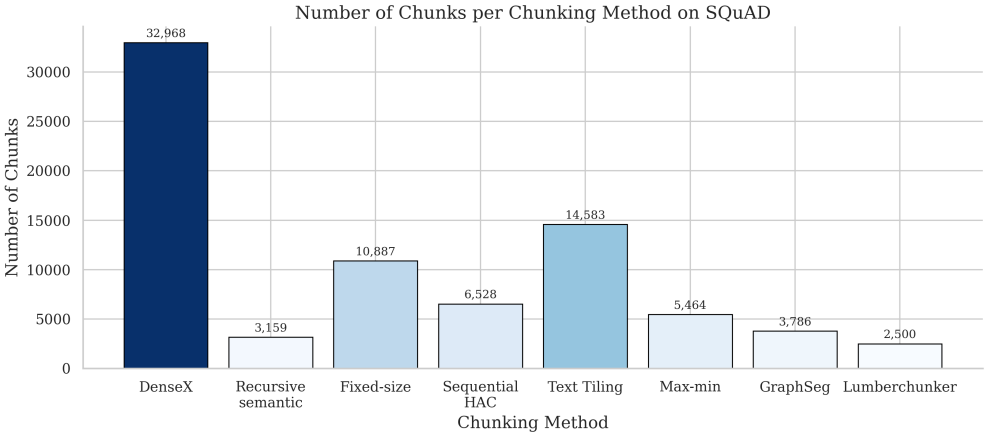

Fig. 1. Number of chunks per chunking method on SQuAD dataset.

Table 4. LLM-as-a-judge score in answer generation task

Dataset	DenseX	Recursive semantic	Fixed-size	Sequential HAC	Max-min	GraphSeg	Lumberchunker
GutenQA	-	**3.65**	3.30	2.50	3.05	3.41	-
GutenQA merged	-	**3.59**	3.25	2.48	3.02	-	-
LiteraryQA	-	**2.10**	1.93	1.45	1.65	-	-
PoQuAD	3.77	4.24	**4.25**	3.84	4.07	4.24	4.21
PoQuAD merged	3.09	4.09	4.18	3.87	4.08	-	**4.28**
SQuAD	4.44	4.56	4.57	4.44	4.53	**4.58**	4.57
TriviaQA	3.99	**4.37**	4.32	4.09	4.22	4.30	-
TriviaQA merged	3.84	**4.32**	4.26	4.16	4.22	-	-
Average	3.83	3.86	3.76	3.35	3.61	4.13	**4.35**

Figure 4 shows the number of chunks per chunking method on SQuAD dataset. Methods with fewer but more coherent chunks (e.g., GraphSeg and Lumberchunker) achieve the strongest Recall@10 and Accuracy@5, despite producing far fewer chunks than over-segmented approaches like DenseX or TextTiling. Higher chunk counts in simpler baselines do not consistently translate into better results, indicating a limited benefit from increased segmentation density alone. Overall, chunk quality and structural coherence are more important than chunk quantity for both Recall@10 and Accuracy@5.

In second experiment, the end-to-end RAG answer generation results in Tab. 4 show that Lumberchunker achieves the highest average score, but this is based only on a limited number of successful runs, which makes the estimate less reliable and prevents strong conclusions about its overall advantage. Excluding this limitation, GraphSeg, Recursive Semantic and DenseX consistently perform among the best methods. Fixed-size chunking remains a solid baseline that is competitive with more complex approaches, while Max-min performs below these methods but still outperforms Sequential HAC, which is the weakest method overall. Overall, the findings indicate that although advanced methods can provide marginal gains, simpler approaches remain highly competitive, and any apparent advantage of LLM-based chunking should be interpreted cautiously due to incomplete coverage.

5. Conclusion and Future Work

Our experiments show that chunking in RAG systems is a much more difficult and fragile problem than is often assumed. Many methods fail in practice due to timeouts, memory issues, or poor scalability, and only a subset (Recursive Semantic, Fixed-size, Sequential HAC, Max-min) consistently completes processing across datasets. Results also vary strongly depending on document type and implementation constraints, and the execution time differences between methods are substantial. This indicates that chunking performance cannot be assessed without considering robustness and efficiency, not only retrieval metrics. Future work should focus on further systematic and fair evaluation of chunking methods across diverse datasets and document types as well as extending evaluations to multiple embedding models and reranking strategies to assess generalisability across retrieval components. Comparisons in literature are often limited and potentially biased toward specific scenarios. A unified taxonomy that includes robustness, scalability, and computational cost is needed. Overall, chunking should be treated as a core research problem in RAG systems rather than a simple preprocessing step. Only such a comprehensive and standardized comparison will make it possible to answer whether chunking methods have a meaningful impact on RAG quality, or whether observed differences are mainly driven by dataset selection and implementation effects.

Acknowledgements

Financed by: CLARIN-PL project financed as part of the investment: ”CLARIN ERIC – European Research Infrastructure Consortium: Common Language Resources and Technology Infrastructure (2024-2026), funded by the Polish Ministry of Science and Higher Education (2024/WK/01).

Appendix A. Datasets

The goal was to ensure a meaningful and fair evaluation of chunking strategies across diverse domains and document structures. First, document length plays a critical role in chunking evaluation. Chunking is inherently irrelevant for short texts, as there is little to no need for segmentation. Consequently, many widely used QA datasets were excluded or deemphasized because their documents are too short to meaningfully benefit from chunking. Second, most existing datasets do not provide explicit annotations of *golden chunks*, i.e., text segments that optimally contain the information needed to answer a query. To address this limitation, we selected datasets that mostly include answer annotations grounded in the source documents (e.g., answer spans, supporting passages, or evidence labels). These annotations allow us to approximate gold chunks indirectly by identifying regions of the text that contain relevant information. The datasets used in our study span multiple domains and levels of complexity:

- *Narrative and literary datasets:*
  GutenQA [6], *LiteraryQA* [1], and *NovelQA* [26] provide long-form narrative texts, often derived from books (e.g., Project Gutenberg). These datasets feature document lengths ranging from several thousand characters to entire novels. They are particularly suitable for evaluating chunking in story-based QA, where relevant information may be distributed across distant parts of the text.
- *Scientific and structured documents:*
  Qasper [5] focuses on scientific papers with structured sections (e.g., headings and paragraphs). It includes explicit evidence annotations, making it valuable for evaluating chunking methods that leverage document structure (e.g., section-aware or hierarchical chunking).
- *Open-domain and multi-domain QA datasets:*
  TriviaQA [11], *SQuAD* [21], *PoQuAD* [24] and *Natural Questions* [15] a wide range of topics and sources.

In summary, dataset selection for chunking evaluation requires careful balancing of three factors: (1) sufficiently long documents, (2) availability of answer-grounded annotations, and (3) diversity of domains and structures. Despite the lack of explicit *golden chunks* in several datasets, the chosen datasets provide a practical foundation for assessing chunking strategies through their impact on retrieval performance. The selected datasets statistics are presented in Table A.5.

Table A.5. Datasets statistics.

Dataset	Documents	Min document length	Max document length	Avg document length
GutenQA	36,917	4	32,767	1,814
GutenQA merged	1	67,058,241	67,058,241	67,058,241
LiteraryQA	138	1,459	1,833,987	411,471
Natural Questions	300	82	215,119	45,634
NovelQA	60	322,441	6,839,619	1,007,786
PoQuAd	1,449	501	3,860	922
Qasper	416	314	2,003	1,014
SQuAD	100	3,715	92,637	29,971
TriviaQA	1,000	113	430,874	14,239
TriviaQA merged	1	14,241,172	14,241,172	14,241,172

References

[1] Bonomo, T., Gioffré, L., Navigli, R., 2025. LiteraryQA: Towards effective evaluation of long-document narrative QA, in: Christodoulopoulos, C., Chakraborty, T., Rose, C., Peng, V. (Eds.), Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Suzhou, China. pp. 34086–34107. URL: https://aclanthology.org/2025.emnlp-main.1729/, doi:10.18653/v1/2025.emnlp-main.1729.

[2] Chase, H., 2022. Langchain. https://github.com/langchain-ai/langchain. Accessed: 2025-05-20.

[3] Chen, T., Wang, H., Chen, S., Yu, W., Ma, K., Zhao, X., Zhang, H., Yu, D., 2024. Dense X retrieval: What retrieval granularity should we use?, in: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Miami, Florida, USA. pp. 15159–15177. URL: https://aclanthology.org/2024.emnlp-main.845/, doi:10.18653/v1/2024.emnlp-main.845.

[4] Dadas, S., Perełkiewicz, M., Poświata, R., 2024. PIRB: A comprehensive benchmark of Polish dense and hybrid text retrieval methods, in: Calzolari, N., Kan, M.Y., Hoste, V., Lenci, A., Sakti, S., Xue, N. (Eds.), Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), ELRA and ICCL, Torino, Italia. pp. 12761–12774. URL: https://aclanthology.org/2024.lrec-main.1117/.

[5] Dasigi, P., Lo, K., Beltagy, I., Cohan, A., Smith, N.A., Gardner, M., 2021. A dataset of information-seeking questions and answers anchored in research papers. URL: https://arxiv.org/abs/2105.03011, arXiv:2105.03011.

[6] Duarte, A.V., Marques, J.D., Graça, M., Freire, M., Li, L., Oliveira, A.L., 2024. LumberChunker: Long-form narrative document segmentation, in: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2024, Association for Computational Linguistics, Miami, Florida, USA. pp. 6473–6486. URL: https://aclanthology.org/2024.findings-emnlp.377/, doi:10.18653/v1/2024.findings-emnlp.377.

[7] Gomez-Cabello, C.A., Prabha, S., Haider, S.A., Genovese, A., Collaco, B.G., Wood, N.G., Bagaria, S., Forte, A.J., 2025. Comparative evaluation of advanced chunking for retrieval-augmented generation in large language models for clinical decision support. Bioengineering 12. URL: https://www.mdpi.com/2306-5354/12/11/1194, doi:10.3390/bioengineering12111194.

[8] Günther, M., Mohr, I., Williams, D.J., Wang, B., Xiao, H., 2025. Late chunking: Contextual chunk embeddings using long-context embedding models. URL: https://arxiv.org/abs/2409.04701, arXiv:2409.04701.

[9] Hearst, M.A., 1997. Text tiling: Segmenting text into multi-paragraph subtopic passages. Computational Linguistics 23. URL: https://aclanthology.org/J97-1003.pdf.

[10] Jain, A., Aggarwal, P., Saladi, A., 2025. AutoChunker: Structured text chunking and its evaluation, in: Rehm, G., Li, Y. (Eds.), Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track), Association for Computational Linguistics, Vienna, Austria. pp. 983–995. URL: https://aclanthology.org/2025.acl-industry.69/, doi:10.18653/v1/2025.acl-industry.69.

[11] Joshi, M., Choi, E., Weld, D., Zettlemoyer, L., 2017. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension, in: Barzilay, R., Kan, M.Y. (Eds.), Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vancouver, Canada. pp. 1601–1611. URL: https://aclanthology.org/P17-1147/, doi:10.18653/v1/P17-1147.

[12] Kamradt, G., 2024. 5 levels of text splitting: Semantic chunking. https://github.com/FullStackRetrieval-com/RetrievalTutorials. Tutorial and Reference Implementation.

[13] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t., 2020. Dense passage retrieval for open-domain question answering, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 6769–6781.

[14] Kiss, A., et al., 2025. Max-min semantic chunking. Discover Computing 28. URL: https://link.springer.com/journal/44227. article number: 117.

[15] Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., Toutanova, K., Jones, L., Kelcey, M., Chang, M.W., Dai, A.M., Uszkoreit, J., Le, Q., Petrov, S., 2019. Natural questions: A benchmark for question answering research. Transactions of the Association for Computational Linguistics 7, 452–466. URL: https://aclanthology.org/Q19-1026/,

doi:10.1162/tacl_a_00276.

[16] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks, in: Proceedings of the 34th International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA.

[17] Lu, W., Chen, K., Qiao, R., Sun, X., 2026. Hichunk: Evaluating and enhancing retrieval-augmented generation with hierarchical chunking. URL: https://openreview.net/forum?id=yCyv2Ij3bS.

[18] Pavlu, V., Rajput, S., Golbus, P.B., Aslam, J.A., 2012. Ir system evaluation using nugget-based test collections, in: Proceedings of the Fifth ACM International Conference on Web Search and Data Mining, Association for Computing Machinery, New York, NY, USA. p. 393–402. URL: https://doi.org/10.1145/2124295.2124343, doi:10.1145/2124295.2124343.

[19] Pradeep, R., Thakur, N., Upadhyay, S., Campos, D., Craswell, N., Soboroff, I., Dang, H.T., Lin, J., 2025. The great nugget recall: Automating fact extraction and rag evaluation with large language models, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, Association for Computing Machinery, New York, NY, USA. p. 180–190. URL: https://doi.org/10.1145/3726302.3730090, doi:10.1145/3726302.3730090.

[20] Qu, R., Tu, R., Bao, F.S., 2025. Is semantic chunking worth the computational cost?, in: Chiruzzo, L., Ritter, A., Wang, L. (Eds.), Findings of the Association for Computational Linguistics: NAACL 2025, Association for Computational Linguistics, Albuquerque, New Mexico. pp. 2155–2177. URL: https://aclanthology.org/2025.findings-naacl.114/, doi:10.18653/v1/2025.findings-naacl.114.

[21] Rajpurkar, P., Zhang, J., Lopyrev, K., Liang, P., 2016. Squad: 100,000+ questions for machine comprehension of text. URL: https://arxiv.org/abs/1606.05250, arXiv:1606.05250.

[22] Reimers, N., Gurevych, I., 2019. Sentence-bert: Sentence embeddings using siamese bert-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 3982–3992.

[23] Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E., Schärli, N., Zhou, D., 2023. Large language models can be easily distracted by irrelevant context, in: Proceedings of the 40th International Conference on Machine Learning, JMLR.org.

[24] Tuora, R., Zwierzchowska, A., Zawadzka-Paluektau, N., Klamra, C., Kobyliński, L., 2023. Poquad - the polish question answering dataset - description and analysis, in: Proceedings of the 12th Knowledge Capture Conference 2023, Association for Computing Machinery, New York, NY, USA. p. 105–113. URL: https://doi.org/10.1145/3587259.3627548, doi:10.1145/3587259.3627548.

[25] Verma, P., 2025. S2 chunking: A hybrid framework for document segmentation through integrated spatial and semantic analysis. URL: https://arxiv.org/abs/2501.05485, arXiv:2501.05485.

[26] Wang, C., Ning, R., Pan, B., Wu, T., Guo, Q., Deng, C., Bao, G., Hu, X., Zhang, Z., Wang, Q., Zhang, Y., 2025a. Novelqa: Benchmarking question answering on documents exceeding 200k tokens. URL: https://arxiv.org/abs/2403.12766, arXiv:2403.12766.

[27] Wang, H., Zhang, D., Li, J., Feng, Z., Zhang, F., 2025b. Entropy-optimized dynamic text segmentation and rag-enhanced llms for construction engineering knowledge base. Applied Sciences 15. URL: https://www.mdpi.com/2076-3417/15/6/3134, doi:10.3390/app15063134.

[28] Wang, X., Wang, Z., Gao, X., Zhang, F., Wu, Y., Xu, Z., Shi, T., Wang, Z., Li, S., Qian, Q., Yin, R., Lv, C., Zheng, X., Huang, X., 2024. Searching for best practices in retrieval-augmented generation, in: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Miami, Florida, USA. pp. 17716–17736. URL: https://aclanthology.org/2024.emnlp-main.981/, doi:10.18653/v1/2024.emnlp-main.981.

[29] Wang, Z., Araki, J., Jiang, Z., Parvez, M.R., Neubig, G., 2023. Learning to filter context for retrieval-augmented generation. URL: https://arxiv.org/abs/2311.08377, arXiv:2311.08377.

[30] Wang, Z., Gao, C., Xiao, C., Huang, Y., Si, S., Luo, K., Bai, Y., Li, W., Duan, T., Lv, C., Lu, G., Chen, G., Qi, F., Sun, M., 2025c. Document segmentation matters for retrieval-augmented generation, in: Findings of the Association for Computational Linguistics: ACL 2025, Association for Computational Linguistics, Vienna, Austria. pp. 8063–8075. URL: https://aclanthology.org/2025.findings-acl.422/, doi:10.18653/v1/2025.findings-acl.422.

[31] Zhang, Y., Zhao, X., Wang, Z.Z., Yang, C., Wei, J., Wu, T., 2025. cAST: Enhancing code retrieval-augmented generation with structural chunking via abstract syntax tree, in: Christodoulopoulos, C., Chakraborty, T., Rose, C., Peng, V. (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2025, Association for Computational Linguistics, Suzhou, China. pp. 8106–8116. URL: https://aclanthology.org/2025.findings-emnlp.430/, doi:10.18653/v1/2025.findings-emnlp.430.

[32] Zhao, J., Ji, Z., Fan, Z., Wang, H., Niu, S., Tang, B., Xiong, F., Li, Z., 2025a. MoC: Mixtures of text chunking learners for retrieval-augmented generation system, in: Che, W., Nabende, J., Shutova, E., Pilehvar, M.T. (Eds.), Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria. pp. 5172–5189. URL: https://aclanthology.org/2025.acl-long.258/, doi:10.18653/v1/2025.acl-long.258.

[33] Zhao, J., Ji, Z., Feng, Y., Qi, P., Niu, S., Tang, B., Xiong, F., Li, Z., 2025b. Meta-chunking: Learning text segmentation and semantic completion via logical perception. URL: https://arxiv.org/abs/2410.12788, arXiv:2410.12788.

[34] Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I., 2023. Judging llm-as-a-judge with mt-bench and chatbot arena, in: Proceedings of the 37th International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA.

[35] Zhong, Z., Liu, H., Cui, X., Zhang, X., Qin, Z., 2025. Mix-of-granularity: Optimize the chunking granularity for retrieval-augmented generation, in: Rambow, O., Wanner, L., Apidianaki, M., Al-Khalifa, H., Eugenio, B.D., Schockaert, S. (Eds.), Proceedings of the 31st International Conference on Computational Linguistics, Association for Computational Linguistics, Abu Dhabi, UAE. pp. 5756–5774. URL: https://aclanthology.org/2025.coling-main.384/.

[36] Zhou, Y., Wang, S., Koopman, B., Zuccon, G., 2026. Beyond chunk-then-embed: A comprehensive taxonomy and evaluation of document chunking strategies for information retrieval. URL: https://arxiv.org/abs/2602.16974, arXiv:2602.16974.